

Control Final Project

Finally, update the state memory:



Milestone1: Topic Discovery & Graph Inspection

```
src
├── Control_Task
│   └── Control_Project
│       ├── assets
│       │   └── demo.gif
│       ├── bicycle_control
│       │   ├── bicycle_control
│       │   │   ├── controller_node.py
│       │   │   ├── __init__.py
│       │   │   ├── lateral_pid.py
│       │   │   ├── longitudinal_pid.py
│       │   │   ├── mpc.py
│       │   │   ├── pure_pursuit.py
│       │   │   ├── teleop_bridge.py
│       │   │   └── velocity_profiler.py
│       │   ├── package.xml
│       │   ├── resource
│       │   │   └── bicycle_control
│       │   ├── setup.cfg
│       │   ├── setup.py
│       │   └── test
│       │       ├── test_copyright.py
│       │       └── test_flake8.py
```

```

|       |— test_lateral_pid.py
|       |— test_longitudinal_pid.py
|       |— test_mpc.py
|       |— test_pep257.py
|       |— test_pure_pursuit.py
|       |— test_velocity_profiler.py
|— bicycle_sim
|   |— bicycle_sim
|   |   |— bicycle_model.py
|   |   |— __init__.py
|   |   |— sim_node.py
|   |— launch
|   |   |— bicycle.rviz
|   |   |— bicycle_sim.launch.py
|   |   |— kinematic_bicycle.launch.py
|   |— package.xml
|   |— resource
|   |   |— bicycle_sim
|   |— setup.cfg
|   |— setup.py
|   |— test
|   |   |— test_bicycle_model.py
|   |   |— test_copyright.py
|   |   |— test_flake8.py
|   |   |— test_pep257.py
|   |— urdf
|   |   |— racecar.urdf
|   |   |— racecar.xacro
|— LICENSE
|— README.md
|— rosgraph.dot
|— rosgraph.svg
|— track_environment
|   |— package.xml
|   |— resource
|   |   |— track_environment

```

```

├── setup.cfg
├── setup.py
├── test
│   ├── test_copyright.py
│   ├── test_flake8.py
│   ├── test_pep257.py
│   └── test_track.py
├── track_environment
│   ├── __init__.py
│   ├── lap_analyzer.py
│   ├── path_gen.py
│   └── track.py
└── tracks
    ├── centerline_0.csv
    ├── global_waypoints.json
    └── random_track0.csv

```

Before we start anything it is important to understand our workspace:

src/Control_Task/Control_Project/

```

├── bicycle_sim/      # Package 1: The Virtual World & Vehicle Physics
├── track_environment/ # Package 2: The Racetrack, Path & Telemetry Judge
└── bicycle_control/  # Package 3: The Brains (Controllers)

```

Package 1: `bicycle_sim` (Vehicle Physics & Simulator)

This package simulates the physical car moving across a 2D surface.

- `bicycle_model.py` :
 - **Role:** Houses the vehicle's physics engine (`Car` class).
 - **Inputs:** Subscribes to `/throttle` and `/steer`.
 - **Outputs:** Publishes `/state` (`nav_msgs/msg/Odometry`), `/joint_states` (wheel animations), and coordinate transforms via `/tf`.
- `sim_node.py` :
 - **Role:** The entrypoint script that instantiates and runs the `Car` node from `bicycle_model.py`.
- `launch/bicycle_sim.launch.py` :
 - **Role:** Master launch file. Starts `sim_node` , `path_gen` , `lap_analyzer` , RViz, and

whichever controller you select via `controller:=<mode>`.

- `urdf/racecar.urdf` & `racecar.xacro` :
 - **Role:** 3D CAD definition of the chassis, hinges, and wheels rendered in RViz.

Package 2: `track_environment` (Track Layout & Performance Judge)

This package loads track waypoints, publishes the path, and computes benchmarking metrics.

- `tracks/centerline_0.csv` :
 - **Role:** The racetrack coordinate database (1,000 centerline (x, y) waypoints with precomputed curvature κ and arc length s).
- `tracks/random_track0.csv` :
 - **Role:** Contains colored cone coordinates (blue/yellow/orange) outlining the track boundaries.
- `track.py` :
 - **Role:** Pure Python parser that reads the CSV files and handles path search paths.
- `path_gen.py` :
 - **Role:** Publishes the racetrack waypoints as a `nav_msgs/msg/Path` on `/path` and boundaries on `/track_bounds`.
- `lap_analyzer.py` :
 - **Role:** The referee and telemetry logger.
 - **Inputs:** Subscribes to `/path` and `/state`.
 - **Outputs:** Calculates Cross-Track Error (CTE), heading error, and lap times. Publishes `/telemetry/*`, `/lap/metrics`, and RViz visual markers.

Package 3: `bicycle_control` (Autonomous & Manual Controllers)

This package contains all control logic. The nodes read the car's current pose from `/state` and the reference line from `/path`, then compute `/throttle` and `/steer` commands.

- `teleop_bridge.py` (Milestone 3):
 - Converts keyboard velocity commands from `/cmd_vel` into `/throttle` and `/steer` with a safety watchdog timer.
- `longitudinal_pid.py` (Milestone 4):
 - Cruise control PID loop that modulates `/throttle` `[-1.0, 1.0]` with anti-windup clamping to regulate vehicle speed.

- `velocity_profiler.py` (Milestone 5.1):
 - Computes curvature-safe target speed profiles
- `lateral_pid.py` (Milestone 5.2):
 - Reactive feedback controller steering the front wheels from Cross-Track Error and heading error.
- `pure_pursuit.py` (Milestone 5.3):
 - Geometric lookahead controller computing steering angles based on circular arc preview.
- `mpc.py` (Milestone 5.4):
 - Model Predictive Controller that solves constrained optimization problems over a prediction horizon.
- `controller_node.py` :
 - Unified wrapper node that imports and runs your chosen lateral and longitudinal controllers together.
- When the simulation is **not running**, `ros2 topic list` only displays `/parameter_events` and `/rosout`.
- When the simulation **is launched**, nodes start advertising topics, sending messages, and building the communication network.

After building and sourcing we can start the simulation through:

```
ros2 launch bicycle_sim bicycle_sim.launch.py controller:=teleop rviz:=true analyzer:=true
```

After doing so we notice that we started the simulator node, path generator, analyzer node, robot state publisher, and RViz window.

In a second terminal we can run: `ros2 topic list`

and we will get all the working topics rn :

```
youssef_ahmed@DESKTOP-OK79IR3:~/ros2_ws/ros2_ws$ ros2 topic list
/clicked_point
/cmd_vel
/goal_pose
/initialpose
/joint_states
/lap/metrics
/lap/visualization
```

```
/parameter_events
/path
/robot_description
/rosout
/state
/steer
/telemetry/cte
/telemetry/heading_err_deg
/telemetry/lap_time
/telemetry/speed
/tf
/tf_static
/throttle
/track_bounds
```

But we see only the topics not who is publishing to who and who is sub to who. Also what is the msg type they communicate through all of that we can see through: `ros2 topic info <topic>`.

For Example:

```
ros2 topic info /state -v
Type: nav_msgs/msg/Odometry

Publisher count: 1

Node name: kinematic_bicycle
Node namespace: /
Topic type: nav_msgs/msg/Odometry
Endpoint type: PUBLISHER
GID: 01.0f.0d.78.f6.0f.10.99.00.00.00.00.00.12.03.00.00.0
0.00.00.00.00.00
QoS profile:
  Reliability: RELIABLE
  History (Depth): UNKNOWN
```

Durability: VOLATILE
Lifespan: Infinite
Deadline: Infinite
Liveliness: AUTOMATIC
Liveliness lease duration: Infinite

Subscription count: 1

Node name: lap_analyzer
Node namespace: /
Topic type: nav_msgs/msg/Odometry
Endpoint type: SUBSCRIPTION
GID: 01.0f.0d.78.fa.0f.2b.ae.00.00.00.00.00.00.12.04.00.00.00.00.00.00.00
QoS profile:
Reliability: RELIABLE
History (Depth): UNKNOWN
Durability: VOLATILE
Lifespan: Infinite
Deadline: Infinite
Liveliness: AUTOMATIC
Liveliness lease duration: Infinite

Same with actuator topics like steering and throttling:

```
ros2 topic info /throttle -v  
ros2 topic info /steer -v
```

Type: std_msgs/msg/Float32

Publisher count: 1

Node name: teleop_bridge

Node namespace: /
Topic type: std_msgs/msg/Float32
Endpoint type: PUBLISHER
GID: 01.0f.0d.78.fe.0f.d3.a6.00.00.00.00.00.00.11.03.00.00.00.00.00.00.00.00.00.00
QoS profile:
Reliability: RELIABLE
History (Depth): UNKNOWN
Durability: VOLATILE
Lifespan: Infinite
Deadline: Infinite
Liveliness: AUTOMATIC
Liveliness lease duration: Infinite

Subscription count: 1

Node name: kinematic_bicycle
Node namespace: /
Topic type: std_msgs/msg/Float32
Endpoint type: SUBSCRIPTION
GID: 01.0f.0d.78.f6.0f.10.99.00.00.00.00.00.00.15.04.00.00.00.00.00.00.00.00.00
QoS profile:
Reliability: RELIABLE
History (Depth): UNKNOWN
Durability: VOLATILE
Lifespan: Infinite
Deadline: Infinite
Liveliness: AUTOMATIC
Liveliness lease duration: Infinite

Type: std_msgs/msg/Float32

Publisher count: 1

Node name: teleop_bridge

```
Node namespace: /
Topic type: std_msgs/msg/Float32
Endpoint type: PUBLISHER
GID: 01.0f.0d.78.fe.0f.d3.a6.00.00.00.00.00.00.12.03.00.00.0
0.00.00.00.00.00
QoS profile:
  Reliability: RELIABLE
  History (Depth): UNKNOWN
  Durability: VOLATILE
  Lifespan: Infinite
  Deadline: Infinite
  Liveliness: AUTOMATIC
  Liveliness lease duration: Infinite
```

Subscription count: 1

```
Node name: kinematic_bicycle
Node namespace: /
Topic type: std_msgs/msg/Float32
Endpoint type: SUBSCRIPTION
GID: 01.0f.0d.78.f6.0f.10.99.00.00.00.00.00.14.04.00.00.0
0.00.00.00.00.00
QoS profile:
  Reliability: RELIABLE
  History (Depth): UNKNOWN
  Durability: VOLATILE
  Lifespan: Infinite
  Deadline: Infinite
  Liveliness: AUTOMATIC
  Liveliness lease duration: Infinite
```

We saw that there are two msg types we found nav_msgs/msg/Odometry and std_msgs/msg/Float32 but what inside them. what are their units?:

```
ros2 interface show nav_msgs/msg/Odometry
```

```
ros2 interface show std_msgs/msg/Float32
```

```
# This represents an estimate of a position and velocity in free space.
```

```
# The pose in this message should be specified in the coordinate frame given by header.frame_id
```

```
# The twist in this message should be specified in the coordinate frame given by the child_frame_id
```

```
# Includes the frame id of the pose parent.
```

```
std_msgs/Header header
```

```
    builtin_interfaces/Time stamp
```

```
        int32 sec
```

```
        uint32 nanosec
```

```
    string frame_id
```

```
# Frame id the pose points to. The twist is in this coordinate frame.
```

```
string child_frame_id
```

```
# Estimated pose that is typically relative to a fixed world frame.
```

```
geometry_msgs/PoseWithCovariance pose
```

```
    Pose pose
```

```
        Point position
```

```
            float64 x
```

```
            float64 y
```

```
            float64 z
```

```
        Quaternion orientation
```

```
            float64 x 0
```

```
            float64 y 0
```

```
            float64 z 0
```

```
            float64 w 1
```

```
    float64[36] covariance
```

```
# Estimated linear and angular velocity relative to child_frame_id.
```

```
geometry_msgs/TwistWithCovariance twist
```

```
    Twist twist
```

```

    Vector3  linear

        float64 x

        float64 y

        float64 z

    Vector3  angular

        float64 x

        float64 y

        float64 z

    float64[36] covariance

# This was originally provided as an example message.

# It is deprecated as of Foxy

# It is recommended to create your own semantically meaningful
# message.

# However if you would like to continue using this please use
# the equivalent in example_msgs.

float32 data

```

Or if we want to make it clearer:

```

pose
├── pose.position

```

- | | — x ← Vehicle Rear-Axle X position (meters)
- | | — y ← Vehicle Rear-Axle Y position (meters)
- | — pose.orientation
- | — z \ ← Quaternion yaw components (used to compute vehicle heading θ)
- | — w /

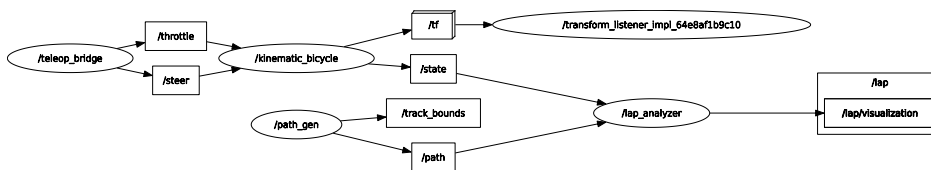
twist

- | — twist.linear
- | — x ← Forward speed v (meters / second)

And for the std_msgs/msg/Float32 it contains only **one single floating-point number** called `float32_data`

Looking at topic lists was not fun lets visualize things:

`ros2 run rqt_graph rqt_graph`



now we can see who sends to whom. like throttle and steering send data to the kinematic_bicycle and kinematic_bicycle updates the transformations and the state

As we visualized the topic nodes, let's visualize our data

`ros2 run plotjuggler plotjuggler`

I use it because it generate memes which is fun and much more flexible and u can plot different topics like this:

Select ROS messages

Topic Selector

Advanced Options

☐ If present, use the timestamp in the field [header.stamp]

Maximum size of arrays:

If larger: ☒ discard ☐ clamp

Select one or multiple topics:

Filter:

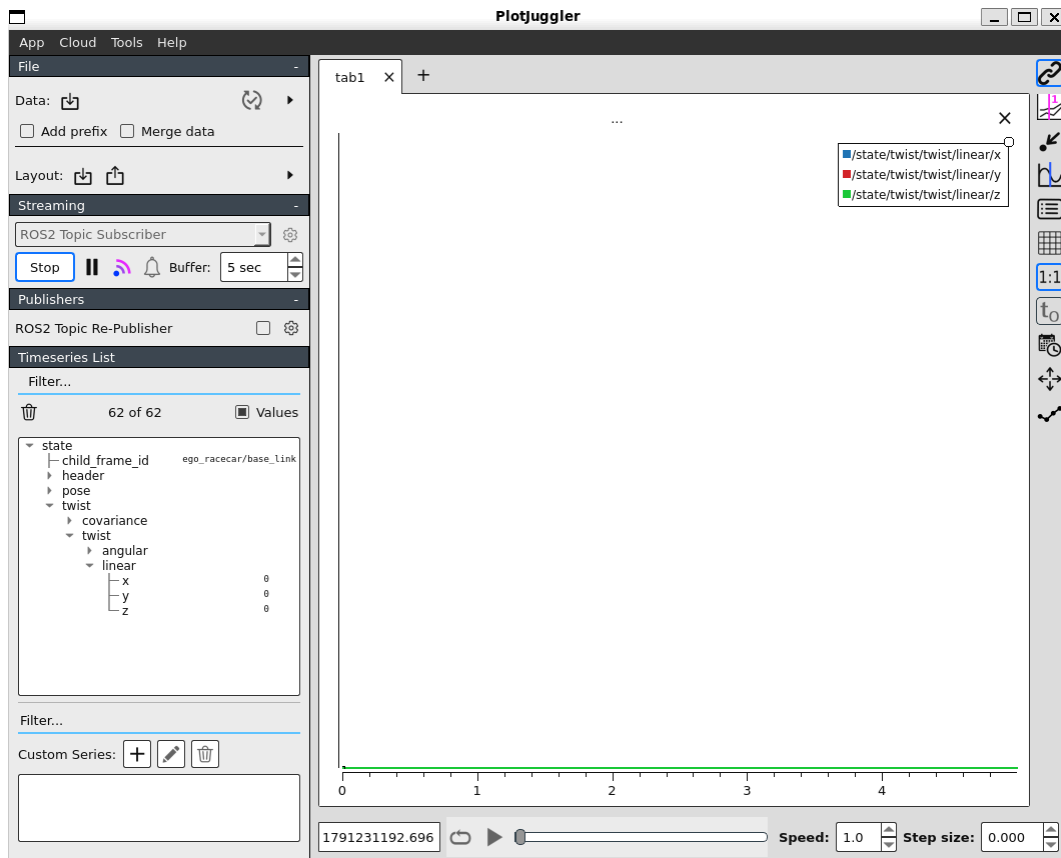
Topic name	Datatype
/clicked_point	geometry_msgs/msg/PointStamped
/cmd_vel	geometry_msgs/msg/Twist
/goal_pose	geometry_msgs/msg/PoseStamped
/initialpose	geometry_msgs/msg/PoseWithCovarianceStamped
/joint_states	sensor_msgs/msg/JointState
/lap/metrics	std_msgs/msg/String
/lap/visualization	visualization_msgs/msg/MarkerArray
/parameter_events	rcl_interfaces/msg/ParameterEvent
/path	nav_msgs/msg/Path
/robot_description	std_msgs/msg/String
/rosout	rcl_interfaces/msg/Log
/state	nav_msgs/msg/Odometry
/steer	std_msgs/msg/Float32
/telemetry/cte	std_msgs/msg/Float32
/telemetry/heading_err_deg	std_msgs/msg/Float32
/telemetry/lap_time	std_msgs/msg/Float32
/telemetry/speed	std_msgs/msg/Float32
/tf	tf2_msgs/msg/TFMessage
/tf_static	tf2_msgs/msg/TFMessage
/throttle	std_msgs/msg/Float32
/track_bounds	visualization_msgs/msg/MarkerArray

Tip: [CTRL+A] select all, [CTRL+SHIFT+A] deselect all

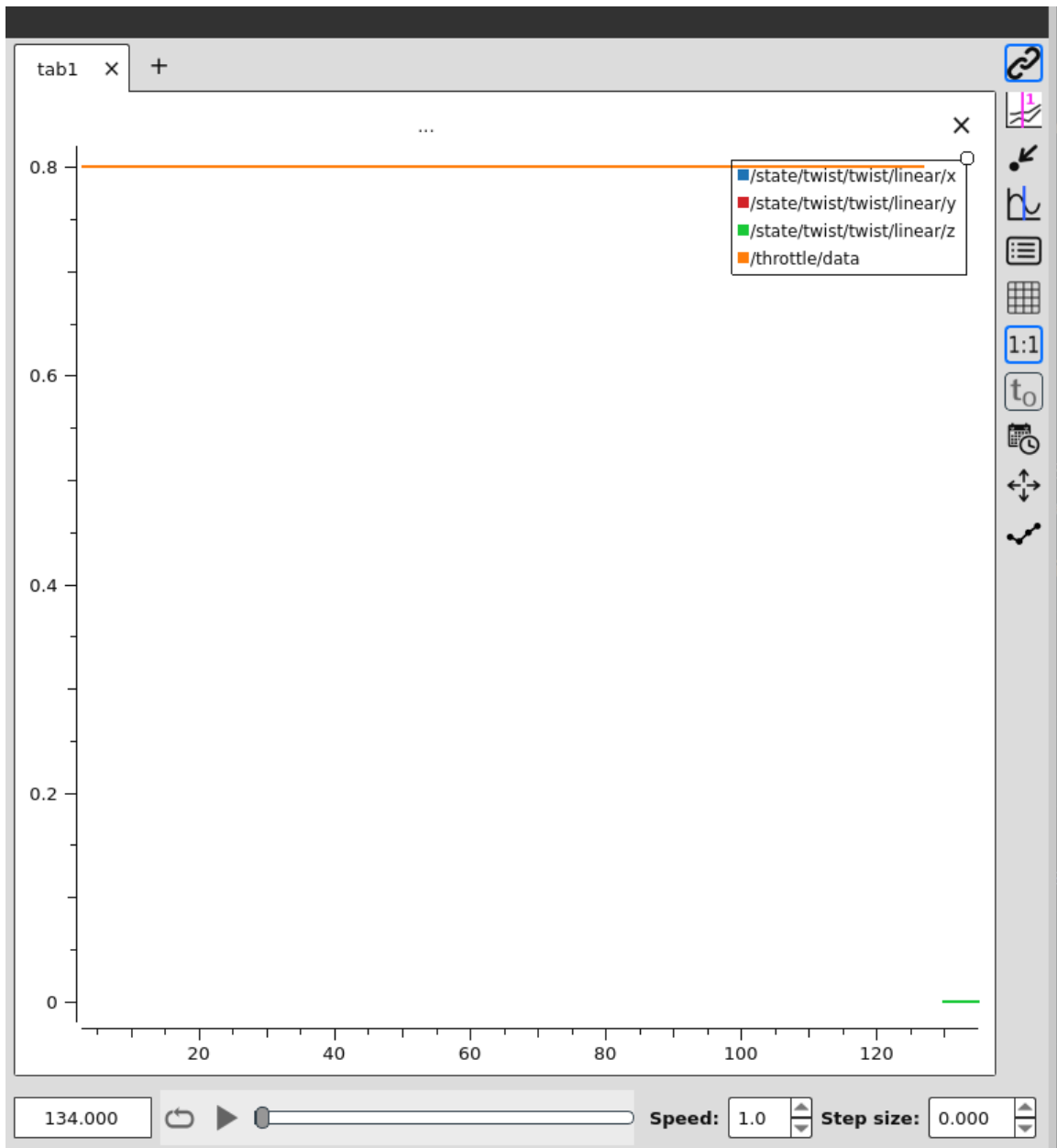
OK

Cancel

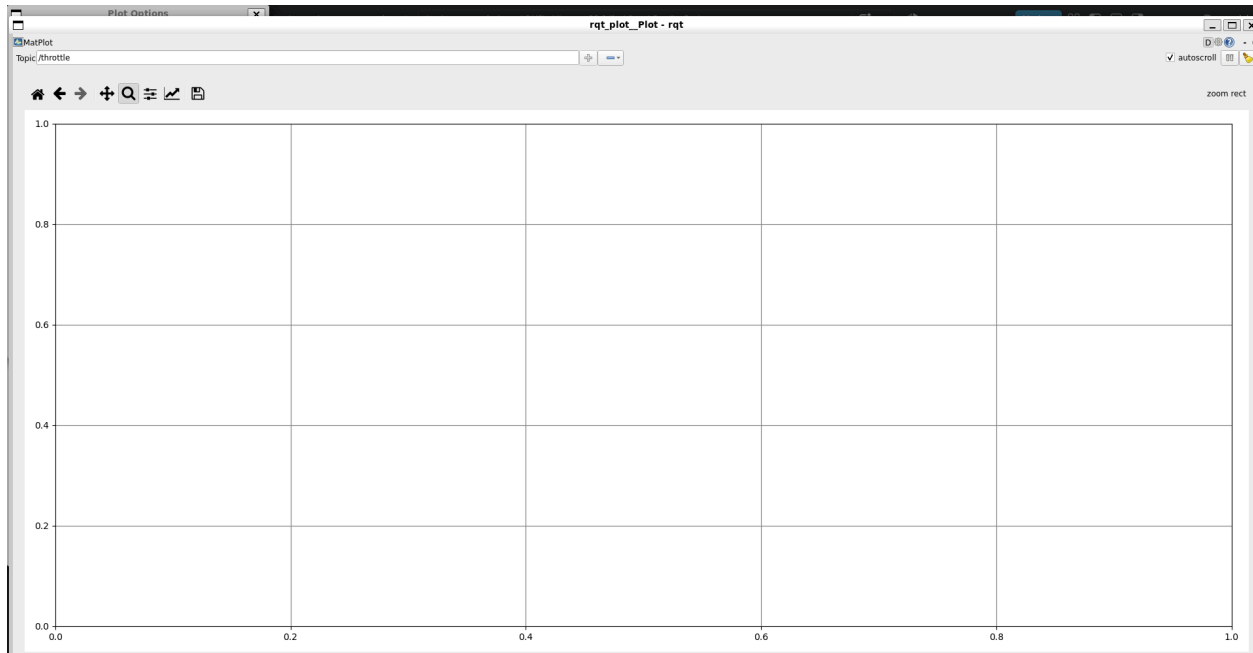
Here we can see the twist linear data being plotted note that it is zero because it doesn't move



And if we added a manual throttle we can see it: `ros2 topic pub --once /throttle std_msgs/msg/Float32 "{data: 0.8}"`



and here is the lame Matplot



Milestone2: Vehicle dynamics

We can see that the mathematical idea was presented in the first block in the `bicycle_model.py` file

```
"""Extended Kinematic Bicycle Model simulator with Drive-by-Wire Powertrain Dynamics.
```

```
    Unlike a basic 3-state kinematic bicycle model where velocity is directly commanded,
```

```
    this extended formulation tracks longitudinal velocity as a dynamic state variable in  $\mathbb{R}^4$ 
```

```
    and uses normalized throttle/braking effort and front steering angle as control inputs in  $\mathbb{R}^2$ .
```

```
State Vector  $x$  in  $\mathbb{R}^4$ :
```

```
    x[0]: x position (m) [rear axle center]
```

```
    x[1]: y position (m) [rear axle center]
```

```
    x[2]: heading theta / yaw (rad)
```

x[3]: longitudinal velocity v (m/s)

Control Inputs u in R^2:

u[0]: normalized throttle/brake command u_throttle in [-1.0, 1.0] via /throttle

[0.0, 1.0] -> Forward motor propulsion effort (scaled by k_a m/s^2)

[-1.0, 0.0) -> Mechanical braking effort (does NOT move the car in reverse)

u[1]: front steering angle delta in radians via /steer (positive = turn left)

"""

$$\mathbf{x} = \begin{bmatrix} x \\ y \\ \theta \\ v \end{bmatrix}$$

x, y: Position of the rear-axle center in meters.

theta: Heading / yaw angle in radians relative to the world frame.

v: Longitudinal velocity in meters per second (m/s)

$$\mathbf{u} = \begin{bmatrix} u_{\text{throttle}} \\ \delta \end{bmatrix}$$

$u_{\text{throttle}} \in [-1.0, 1.0]$: Normalized throttle and brake effort.

KaTeX parse error: Expected '}', got 'EOF' at end of input: ...l in radians.}}

The updated code:

```
def update_x_dot(self):  
    """Computes the state derivative vector x_dot = f(x,  
    u).
```

```

    State Vector self.x (R^4):
        self.x[0]: x position (m) [rear axle center]
        self.x[1]: y position (m) [rear axle center]
        self.x[2]: heading theta / yaw angle (rad, 0 = +x
axis)

        self.x[3]: longitudinal velocity v (m/s)

    Control Input Vector self.u (R^2):
        self.u[0]: normalized throttle/brake command u_th
rottle in [-1.0, 1.0]
        self.u[1]: front steering angle delta in radians
(positive = left)

    Vehicle Physical Parameters:
        self.wheelbase_length (L): 1.25 m
        self.k_a: 4.0 m/s^2 (powertrain acceleration scal
ing gain)

        self.c_drag: 0.005 (aerodynamic drag coefficient)
        self.c_roll: 0.05 (rolling resistance coefficient)

    """

    x_pos, y_pos, theta, v = self.x
    u_throttle, delta = self.u

    # 1. Kinematic planar velocity components
    x_dot = v * math.cos(theta)
    y_dot = v * math.sin(theta)

    # 2. Kinematic yaw rate
    theta_dot = (v / self.wheelbase_length) * math.tan(de
lta)

    # 3. Longitudinal acceleration dynamics with powertra
in and resistance
    a_drive = self.k_a * u_throttle

```

```

        # Resistance forces act only to oppose existing forward velocity
        if v > 0.0:
            a_drag = self.c_drag * (v ** 2)
            a_roll = self.c_roll * v
            v_dot = a_drive - (a_drag + a_roll)
        else:
            # If vehicle is stopped, do not let resistance pull it backward
            v_dot = max(0.0, a_drive)

        self.x_dot = np.array([x_dot, y_dot, theta_dot, v_dot], dtype=np.float64)

    def update_x(self):
        """Integrates state forward using discrete Forward Euler numerical integration."""
        # 1. Euler integration step:  $x[k+1] = x[k] + x\_dot * dt$ 
        self.x = self.x + self.x_dot * self.dt

        # 2. Heading angle wrapping to  $[-\pi, \pi]$ 
        self.x[2] = math.atan2(math.sin(self.x[2]), math.cos(self.x[2]))

        # 3. Longitudinal speed clamping: non-negative (no reverse) and capped at max_speed
        self.x[3] = float(np.clip(self.x[3], 0.0, self.max_speed))

```

1. Position Derivatives ($\dot{x}, \dot{y}$)

The vehicle's position derivatives are determined by its velocity and heading:

$$\dot{x} = v \cos(\theta)$$

$$\dot{y} = v \sin(\theta)$$

2. Heading Derivative ($\dot{\theta}$)

Derived from the rear-axle kinematic bicycle model with wheelbase L :

$$\dot{\theta} = \frac{v}{L} \tan(\delta)$$

where:

- v is the vehicle's longitudinal velocity.
- L is the wheelbase.
- δ is the steering angle.
- θ is the vehicle's heading angle.

3. Longitudinal Acceleration Derivative ($\dot{v}$)

The acceleration is driven by motor thrust and opposed by resistance forces:

$$\dot{v} = a_{\text{powertrain}} - a_{\text{resistance}}$$

Powertrain Effort

The acceleration generated by the powertrain is modeled as:

$$a_{\text{powertrain}} = k_a \cdot u_{\text{throttle}}$$

where:

- k_a is the acceleration gain.
- u_{throttle} is the throttle input.

Resistance Forces

Resistance opposes forward motion when the vehicle is moving ($v > 0$):

$$a_{\text{resistance}} = c_{\text{drag}} \cdot v^2 + c_{\text{roll}} \cdot v$$

where:

- c_{drag} represents aerodynamic drag.
- c_{roll} represents rolling resistance.

Note: If $v = 0$ and $u_{\text{throttle}} \leq 0$, resistance is set to 0 to keep the vehicle at rest.

4. Discrete-Time Forward Euler Integration

The continuous-time state equation can be written as:

$$\mathbf{x}_{k+1} = \mathbf{x}_k + \dot{\mathbf{x}}_k \Delta t$$

Every simulation step, with:

$$\Delta t = 0.1 \text{ s}$$

the simulator updates the vehicle state using Forward Euler integration.

Position Update

$$x_{k+1} = x_k + \dot{x} \cdot \Delta t$$

$$y_{k+1} = y_k + \dot{y} \cdot \Delta t$$

Heading Update

$$\theta_{k+1} = \text{wrap_angle}(\theta_k + \dot{\theta} \cdot \Delta t)$$

Velocity Update

$$v_{k+1} = \text{clamp}(v_k + \dot{v} \cdot \Delta t, 0.0, v_{\text{max}})$$

5. State Update Constraints

Heading Wrapping

The heading angle is normalized to the range:

$$[-\pi, \pi]$$

using:

$$\text{wrap_angle}(\theta) = \text{atan2}(\sin(\theta), \cos(\theta))$$

This prevents the heading angle from growing indefinitely as the vehicle rotates.

Velocity Clamping

The vehicle's velocity is constrained to:

$$0.0 \leq v \leq v_{\max}$$

This means:

- The vehicle cannot drive in reverse through braking, since $v \geq 0$.
- The vehicle cannot exceed its mechanical maximum speed, since $v \leq v_{\max}$.

Summary

The complete vehicle model consists of three continuous-time dynamics:

$$\dot{x} = v \cos(\theta)$$

$$\dot{y} = v \sin(\theta)$$

$$\dot{\theta} = \frac{v}{L} \tan(\delta)$$

$$\dot{v} = k_a u_{\text{throttle}} - (c_{\text{drag}} v^2 + c_{\text{roll}} v)$$

These dynamics are then integrated forward in discrete time using:

$$\mathbf{x}_{k+1} = \mathbf{x}_k + \dot{\mathbf{x}}_k \Delta t$$

with $\Delta t = 0.1$ s, while applying heading wrapping and velocity constraints.

Milestone 3: Key teleportation node

To understand this we need to differ between `cmd_vel` and `throttle`. When you run standard ROS 2 keyboard teleop tools (like `teleop_twist_keyboard`), they publish a generic velocity command on `/cmd_vel` using the message type

`geometry_msgs/msg/Twist`:

- `twist.linear.x`: Target longitudinal forward speed (m/s).
- `twist.angular.z`: Target yaw rate / rotational speed (ω_z in rad/s).

However, the vehicle simulator (`bicycle_model.py`) does **not** accept raw velocity or angular rates directly. It requires drive-by-wire actuator inputs:

- `/throttle` (`std_msgs/msg/Float32` in range `[-1.0, 1.0]`)
- `/steer` (`std_msgs/msg/Float32` in radians, positive = left)

The `teleop_bridge.py` node bridges this difference.

From math to code:

```
def cmd_callback(self, msg: Twist):
    """Translates Twist linear.x to throttle [-1, 1] and
    angular.z into steering (rad)."""
    # Record arrival time for the safety watchdog
    self.last_cmd_time = self.get_clock().now()
```

```

# 1. Linear velocity mapping -> throttle [-1.0, 1.0]
self.target_vel = float(msg.linear.x)
if self.max_linear_vel > 0.0:
    raw_throttle = self.target_vel / self.max_linear_
vel
else:
    raw_throttle = 0.0
self.current_throttle = float(np.clip(raw_throttle, -
1.0, 1.0))

# 2. Angular velocity mapping -> steering angle [rad]
raw_angular = float(msg.angular.z)
if self.max_angular_vel > 0.0:
    steer_fraction = raw_angular / self.max_angular_v
el
    raw_steer = steer_fraction * self.max_steer_rad
else:
    raw_steer = 0.0
self.current_steer = float(np.clip(raw_steer, -self.m
ax_steer_rad, self.max_steer_rad))

def publish_commands(self):
    """Periodically publishes throttle and steering comma
nds at 10 Hz."""
    # 1. Safety Watchdog Check
    elapsed_sec = (self.get_clock().now() - self.last_cmd
_time).nanoseconds * 1e-9
    if elapsed_sec > self.auto_zero_timeout:
        # Timed out (> 0.5s): Zero out all control effort
s
        self.current_throttle = 0.0
        self.current_steer = 0.0
        self.target_vel = 0.0

# 2. Publish commands to vehicle actuators
throttle_msg = Float32()

```

```

throttle_msg.data = float(self.current_throttle)
self.throttle_pub.publish(throttle_msg)

steer_msg = Float32()
steer_msg.data = float(self.current_steer)
self.steer_pub.publish(steer_msg)

```

Note about the code: if `self.max_linear_vel > 0.0`:

```

    raw_throttle = self.target_vel / self.max_linear_vel
else:
    raw_throttle = 0.0

```

I used this approach to avoid errors if the max value is zero I know it is assigned as 5 but for just in case.

Now the math meaning of this code:

$$u_{\text{throttle}} = \text{clip} \left(\frac{v_{\text{cmd}}}{v_{\text{max,linear}}}, -1.0, 1.0 \right)$$

remember that throttle ranges between $[-1.0, 1.0]$. But what does clip mean :

$$\text{clipped_value} = \begin{cases} \text{min_val}, & \text{if val} < \text{min_val} \\ \text{max_val}, & \text{if val} > \text{max_val} \\ \text{val}, & \text{otherwise} \end{cases}$$

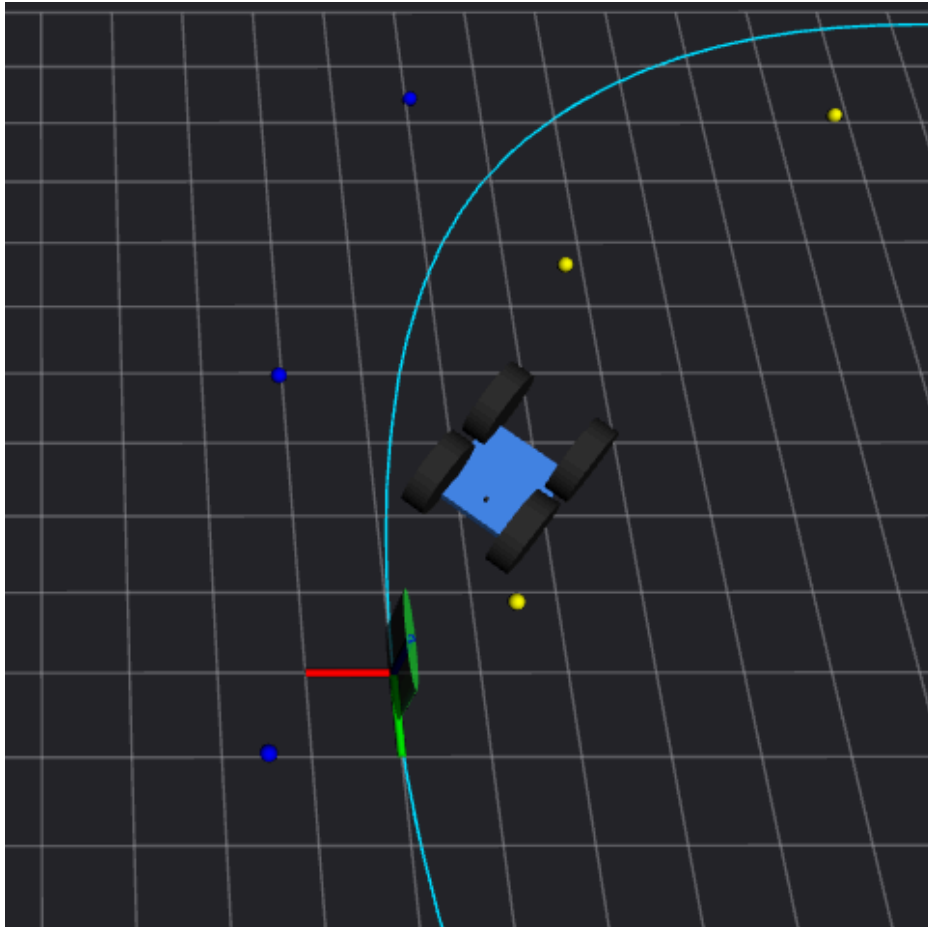
If the number is inside the acceptable range, it is left untouched. If it goes past either edge, it gets pushed back to the boundary limit.

When $v_{\text{cmd}} = 5.0$ m/s and $v_{\text{max, linear}} = 5.0$ m/s, throttle is +1.0 (100% forward motor drive).

When $v_{\text{cmd}} < 0$, it commands negative throttle / mechanical braking effort (down to -1.0).

Same with steering angle:

$$\delta = \text{clip} \left(\frac{\omega_z}{\omega_{\text{max,angular}}} \cdot \delta_{\text{max}}, -\delta_{\text{max}}, \delta_{\text{max}} \right)$$



```
source /opt/ros/humble/setup.bash
source ~/ros2_ws/ros2_ws/install/setup.bash
ros2 run teleop_twist_keyboard teleop_twist_keyboard
```

This node takes keypresses from the keyboard and publishes them as Twist/TwistStamped messages. It works best with a US keyboard layout.

Moving around:

u	i	o
j	k	l

m , .

For Holonomic mode (strafing), hold down the shift key:

U	I	O
J	K	L
M	<	>

t : up (+z)

b : down (-z)

anything else : stop

q/z : increase/decrease max speeds by 10%

w/x : increase/decrease only linear speed by 10%

e/c : increase/decrease only angular speed by 10%

CTRL-C to quit

currently: speed 0.50 turn 1.00

Milestone 4:

First we need to ask ourselves why we need a closed loop control.

$$\dot{v} = k_a \cdot u_{\text{throttle}} - (c_{\text{drag}} v^2 + c_{\text{roll}} v)$$

See this equation. when we set a fixed throttle like 0.5 causes speed to fluctuate based on drag As v increases, opposing drag forces grow quadratically (v^2), so a static throttle command never holds an accurate speed alone. Therefore we need a PID controller.

So let's breakdown the math step by step:

A. Velocity Error

At every control step it compute the tracking error between desired speed and actual speed:

$$e_k = v_{\text{target}} - v_{\text{current}}$$

B. Proportional Term (P)

Provides immediate control response **proportional** to the current error:

$$P_k = K_p \cdot e_k$$

Before explaining I we need to know why P isn't enough:

Suppose the target speed is

$v_{\text{ref}} = 20 \text{ m/s}$

and the car reaches 19 m/s. The remaining error is:

$e = 20 - 19 = 1 \text{ m/s}$

The proportional controller produces

$u_P = K_P e$

and therefore increases the throttle.

However, the car needs some nonzero throttle to maintain 20 m/s because drag and rolling resistance are constantly slowing it down. But if the error reaches zero, then $u_P = K_P(0) = 0$.

This is the problem with P-only control: **the throttle it provides depends on the error, but the car needs nonzero throttle even when the error is zero.**

As a result, the system settles slightly below the target speed, where the remaining error produces enough proportional throttle to balance the drag.

C. Integral Term (I):

Aerodynamic drag ($F_d \propto v^2$) and rolling resistance create a steady-state drag force that the proportional term alone cannot overcome without permanent

steady-state error. The integral term eliminates this offset by accumulating past errors:

D. Derivative Term (D)

Anticipates future error trends and dampens oscillations:

$$D_k = K_d \cdot \frac{e_k - e_{k-1}}{\Delta t}$$

E. Total Output & Actuator Saturation

Sum the terms and clamp the command strictly to the actuator bounds

$$u_{\text{cmd}} = \text{clip}(u_{\text{raw}}, -\text{max_brake}, \text{max_throttle})$$

Finally, update the state memory:

$$e_{k-1} = e_k$$

```
"""
Low-Level Powertrain Cruise Controller (Longitudinal PID).
Regulates vehicle speed via normalized throttle/braking effort.
"""

import numpy as np # noqa: F401

class PIDLongitudinalController:
    """Low-Level Powertrain Cruise Controller / Electronic Speed Control (ESC).

    Translates high-level velocity requests into normalized throttle/brake effort.
    Because physical vehicles experience friction and speed-squared aerodynamic drag,
```

a closed-loop speed regulator is required to maintain target velocity.

"""

```
def __init__(self, kp=1.0, ki=0.2, kd=0.05, dt=0.1,
              max_throttle=1.0, max_brake=1.0, integral_limit=2.0):
```

```
    self.kp = float(kp)
    self.ki = float(ki)
    self.kd = float(kd)
    self.dt = float(dt)
    self.max_throttle = float(max_throttle)
    self.max_brake = float(max_brake)
    self.integral_limit = float(integral_limit)
```

```
    self.integral = 0.0
    self.prev_error = 0.0
    self.first_run = True
```

```
def compute(self, target_vel, current_vel):
    """Computes normalized throttle/braking effort in [-max_brake, max_throttle]."""
```

```
    # If target velocity is zero or car is stopping, cut power immediately
```

```
    if abs(target_vel) < 1e-3:
        self.reset()
```

```
    # If still moving forward, apply gentle brake to stop
```

```
    if current_vel > 0.1:
        return -0.5
    return 0.0
```

```
    # 1. Compute velocity error
```

```
    error = float(target_vel - current_vel)
```

```
    # 2. Proportional term
```

```

    p_term = self.kp * error

    # 3. Integral accumulation with anti-windup clamping
    self.integral += error * self.dt
    self.integral = float(np.clip(self.integral, -self.integral_limit, self.integral_limit))
    i_term = self.ki * self.integral

    # 4. Derivative term (prevent kick on first step)
    if self.first_run:
        d_term = 0.0
        self.first_run = False
    elif self.dt > 0.0:
        derivative = (error - self.prev_error) / self.dt
        d_term = self.kd * derivative
    else:
        d_term = 0.0

    # 5. Total output
    raw_output = p_term + i_term + d_term

    # 6. Actuator clamping [-max_brake, max_throttle]
    output = float(np.clip(raw_output, -self.max_brake, self.max_throttle))

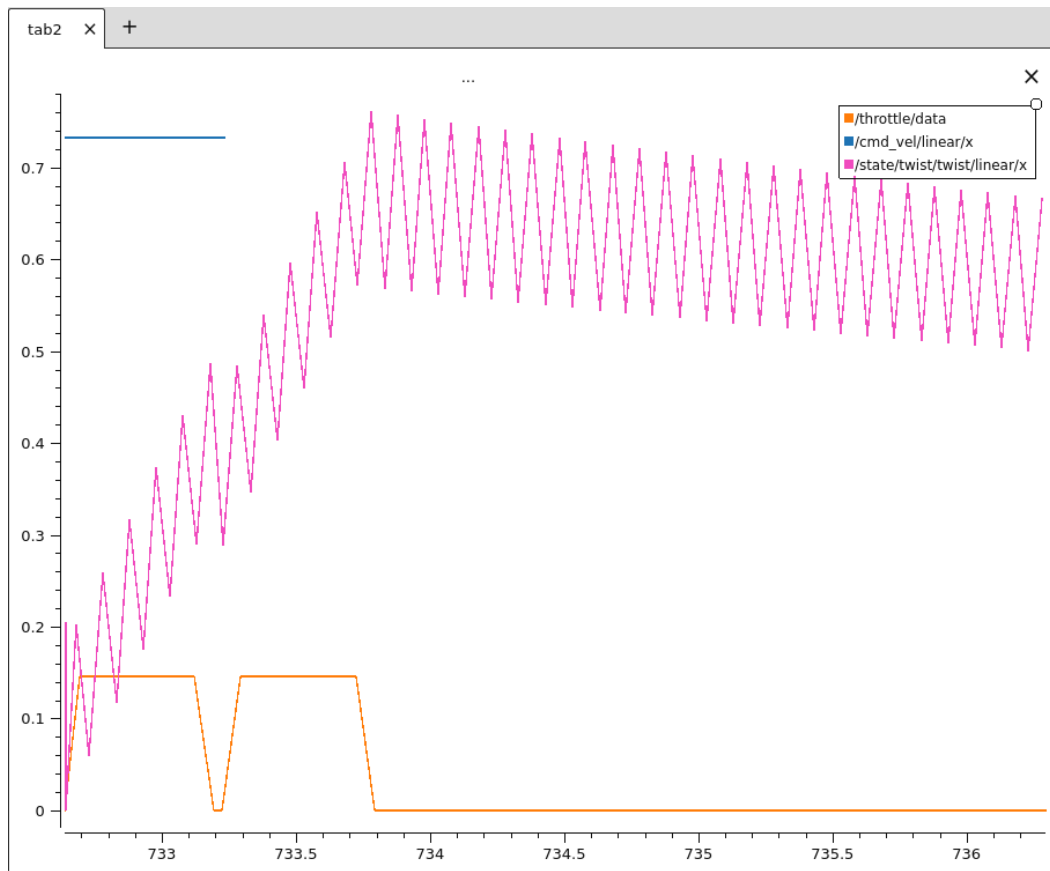
    # 7. Update memory
    self.prev_error = error

    return output

def reset(self):
    """Resets integrator and previous error state."""
    self.integral = 0.0
    self.prev_error = 0.0
    self.first_run = True

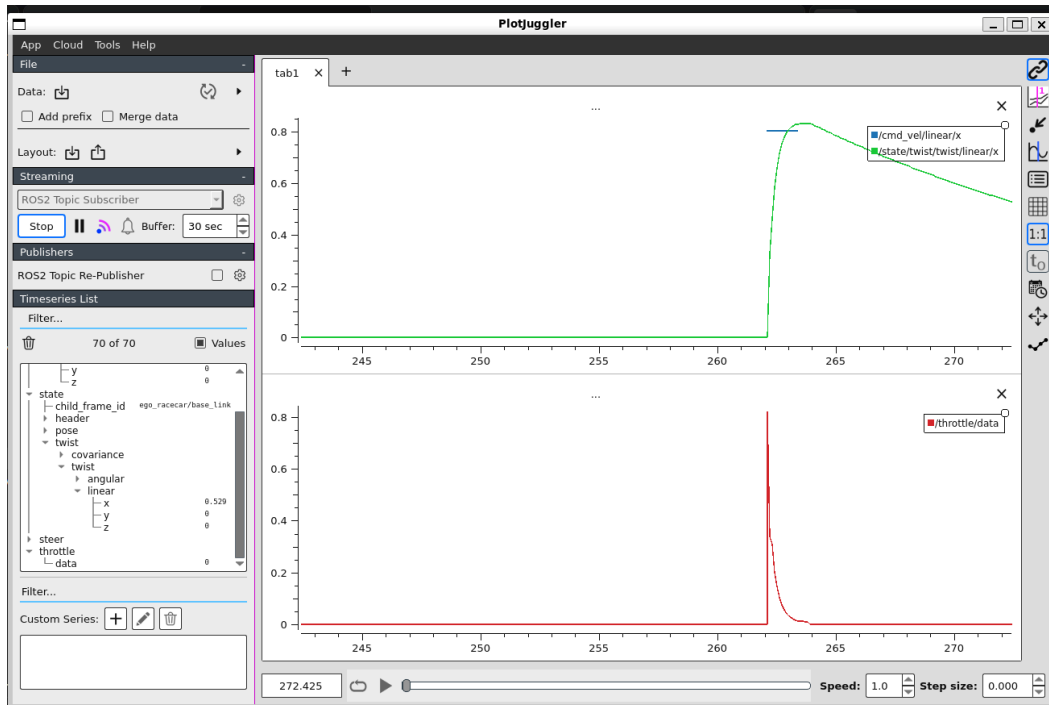
```

Before tuning PID:



After:

Note this is on one throttle only.



```
python3 -m pytest src/Control_Task/Control_Project/bicycle_control/test/test_longitudinal_pid.py -v
ASSED [ 33%]
src/Control_Task/Control_Project/bicycle_control/test/test_longitudinal_pid.py::test_longitudinal_pid_negative_error P
ASSED [ 66%]
src/Control_Task/Control_Project/bicycle_control/test/test_longitudinal_pid.py::test_longitudinal_pid_anti_windup PASS
ED [100%]

===== 3 passed in 0.44s =====
```

Velocity Profiler (Milestone 5.1)

`velocity_profiler.py` answers one question: given the shape of the road at the car's current position, what's the fastest speed the car is allowed to drive?

Going flat-out everywhere doesn't work, because corners cost grip. To follow a bend at speed v with curvature κ , the car needs sideways (lateral) acceleration of: $a_{lat} = v^2 \cdot \kappa$

Kappa is just $1/\text{radius}$: 0 on a straight, bigger for tighter bends. The tires can only provide so much lateral acceleration (in this project the limit is $\text{max_lat_accel} = 5.0 \text{ m/s}^2$). Push past it and the car slides off the track instead of turning. Rearranging the formula gives the fastest safe speed through a corner:

$$v_{\text{max}} = \sqrt{a_{\text{lat_max}} / |\kappa|}$$

Example: a corner with radius 2 m has $\kappa = 0.5$, so $v_{\max} = \sqrt{5 / 0.5} \approx 3.16$ m/s. Tighter corner, lower speed. That's the whole idea.

`compute_target_speed` implements this and wraps it with two practical cases:

- **No curvature data** (κ is None or NaN): we can't do physics without a number, so return the fallback cruise speed (`default_speed` , or `fallback_speed` if the caller passes one). This is the only role of `default_speed` : a conservative "I don't know the road" answer, not a speed cap.
- **Straight** (`abs(kappa) < 1e-5`): no cornering force is needed, so the profiler returns `max_speed` and lets the car use everything it has. (Small confession: my first version returned `default_speed` here because I assumed it was a cruise limit. The unit test expects 8.0 on a straight, and it made the intended semantics clear.)
- **Corner**: apply the formula, then return the smallest of the physics limit, the requested fallback speed, and `max_speed` , i.e. the most restrictive of the three.

One detail worth calling out: the `abs()` on κ . A left turn and a right turn of the same radius need exactly the same grip; the sign of κ only says which way the road bends, not how tight it is.

With the default settings, the behavior looks like this:

Road	κ	returned speed
Straight	0	8.0
R = 20 m bend	0.05	4.0 (cruise fallback caps it)
R = 2 m corner	0.5	≈ 3.16
R = 0.8 m hairpin	1.25	2.0

```
"""
```

```
Target Velocity Profiler based on track curvature.  
Calculates maximum safe cornering speeds subject to lateral acceleration limits.
```

```
"""
```

```
import math # noqa: F401
```

```

class VelocityProfiler:
    """Generates target speed profiles based on track curvature or precomputed data."""

    def __init__(self, default_speed=4.0, max_speed=8.0, max_lat_accel=5.0):
        self.default_speed = default_speed
        self.max_speed = max_speed
        self.max_lat_accel = max_lat_accel

    def compute_target_speed(self, kappa, fallback_speed=None):
        """Calculates curvature-limited velocity:  $v_{\max} = \sqrt{a_{\text{lat\_max}} / |\kappa|}$ ."""
        # Baseline speed used ONLY when curvature data is unavailable
        base_speed = fallback_speed if fallback_speed is not None else self.default_speed

        # No usable curvature data -> conservative fallback (cruise speed)
        if kappa is None or not math.isfinite(kappa):
            return float(min(base_speed, self.max_speed))

        # Straight road (near-zero curvature): no lateral constraint -> full speed
        if abs(kappa) < 1e-5:
            return float(self.max_speed)

        # Corner: physics limit from the lateral acceleration budget,
        # capped by the requested base speed and the global maximum
        safe_cornering_speed = math.sqrt(self.max_lat_accel / abs(kappa))

```

```
return float(min(safe_cornering_speed, base_speed, self.max_speed))
```

```
bash urdf bash ros2_ws x
youssef_ahmed@DESKTOP-OK79IR3:~/ros2_ws/ros2_ws$ source install/setup.bash
python3 -m pytest src/Control_Task/Control_Project/bicycle_control/test/test_velocity_profiler.py -v
D [ 50%]
src/Control_Task/Control_Project/bicycle_control/test/test_velocity_profiler.py::test_velocity_profiler_sharp_turn PASSED
SED [100%]
===== 2 passed in 0.63s =====
```

Lateral PID (Milestone 5.2)

`lateral_pid.py` is the first of the three steering controllers. The question it answers: the car is off the path right now — which way and how hard do I steer to get back?

It works from exactly two numbers:

- **Cross-track error (CTE):** signed distance from the centerline in meters. Positive = car is left of the path.
- **Heading error:** where the nose points minus where the path goes ($\psi_{\text{vehicle}} - \psi_{\text{path}}$). Positive = nose pointing left of the path direction.

The steering law:

$\delta = -(k_p \cdot \text{cte} + k_i \cdot \int \text{cte} \, dt + k_d \cdot d(\text{cte})/dt + k_{\text{yaw}} \cdot \text{heading_err})$, clamped to $\pm 35^\circ$

Why the minus sign is the whole milestone. In this project positive steering turns the car left. Both error terms are positive when the car deviates to the left, and a car left of the path (or pointing left) must steer *right* — negative. So the summed correction is negated once and every term pushes back toward the line. Flip that sign and the car steers *into* its own deviation, off the track; getting this contract right is what "correct sign conventions" in the milestone description is testing.

The four terms each earn their place:

- **P ($k_p = 0.8$)** acts like a spring: twice as far off the line, twice the steering. Gets you back, but on its own it overshoots and oscillates.
- **I ($k_i = 0.02$)** fixes constant bias. With P alone, anything that consistently pushes the car off-line makes it settle *next to* the line with a permanent offset — at zero CTE, P commands zero steer and nothing corrects the residue. The integral accumulates error over time and keeps pushing until the offset is

gone. Its anti-windup clamp (± 1.0) matters during long saturated maneuvers: without it, the accumulator keeps growing while the steering is pinned at 35° , then dumps a huge correction the instant the error crosses zero — that failure mode is called integral windup, the same one we handled in the longitudinal PID.

- **D ($k_d = 0.15$)** is damping: it watches how *fast* the CTE is changing and applies counter-steer before the car actually crosses the line, killing the oscillation P would otherwise sustain.
- **Yaw ($k_{yaw} = 0.5$)** covers what position can't see: a car sitting exactly on the line but pointing 30° off is about to *leave* the line. The heading term corrects the nose before the position error even develops — effectively a second proportional controller on a different error signal.

```
youssef_ahmed@DESKTOP-OK79IR3:~/ros2_ws/ros2_ws$ cd ~/ros2_ws/ros2_ws
source install/setup.bash
python3 -m pytest src/Control_Task/Control_Project/bicycle_control/test/test_lateral_pid.py -v
collected 2 items

src/Control_Task/Control_Project/bicycle_control/test/test_lateral_pid.py::test_lateral_pid_steer_right PASSED [ 50%]
src/Control_Task/Control_Project/bicycle_control/test/test_lateral_pid.py::test_lateral_pid_steer_left PASSED [100%]

===== 2 passed in 0.49s =====
```

The code:

```
"""
High-Level Lateral Steering Controller: Reactive Lateral PID.
Steers based on instantaneous Cross-Track Error (CTE) and Heading Error.
"""

import math
import numpy as np # noqa: F401

class LateralPIDController:
    """Lateral PID steering controller based on Cross-Track Error (CTE) and Heading Error.
```

```
    Commands front wheel steering based on instantaneous lateral offset (cross-track error) and orientation error relative to the nearest path waypoint.
```

```
    """
```

```
    def __init__(self, kp=0.8, ki=0.02, kd=0.15, k_yaw=0.5, dt=0.1,
```

```
                max_steer_rad=math.radians(35.0), integral_limit=1.0):
```

```
        self.kp = kp
```

```
        self.ki = ki
```

```
        self.kd = kd
```

```
        self.k_yaw = k_yaw
```

```
        self.dt = dt
```

```
        self.max_steer_rad = max_steer_rad
```

```
        self.integral_limit = integral_limit
```

```
        self.integral_cte = 0.0
```

```
        self.prev_cte = 0.0
```

```
    def compute_steering(self, cte, heading_err):  
        """Computes front wheel steering angle delta in radians.
```

```
        Args:
```

```
            cte: Signed cross-track error in meters (positive = vehicle is left of path).
```

```
            heading_err: Heading error in radians (psi_vehicle - psi_path).
```

```
        Returns:
```

```
            delta_rad: Commanded front steering angle in radians [-max_steer_rad, max_steer_rad].
```

```
        """
```

```
        # 1. Proportional term for Cross-Track Error
```

```

        p_term = self.kp * cte

        # 2. Integral accumulation with anti-windup clamping
        self.integral_cte += cte * self.dt
        self.integral_cte = float(np.clip(self.integral_cte,
-self.integral_limit, self.integral_limit))
        i_term = self.ki * self.integral_cte

        # 3. Derivative term for damping oscillations
        if self.dt > 0.0:
            d_term = self.kd * (cte - self.prev_cte) / self.d
t
        else:
            d_term = 0.0

        # 4. Heading correction term
        yaw_term = self.k_yaw * heading_err

        # 5. Combine terms (negated to ensure proper correcti
ve steering direction)
        raw_steer = -(p_term + i_term + d_term + yaw_term)

        # 6. Actuator clamping [-max_steer_rad, max_steer_ra
d]
        delta_rad = float(np.clip(raw_steer, -self.max_steer_
rad, self.max_steer_rad))

        # 7. Update memory for the next derivative calculatio
n
        self.prev_cte = cte

        return delta_rad

def reset(self):
    """Resets integrator and previous error state."""

```

```
self.integral_cte = 0.0
self.prev_cte = 0.0
```

Pure Pursuit (Milestone 5.3)

`pure_pursuit.py` is the second steering controller and the first one that looks ahead. The lateral PID reacts to where the car *is*; pure pursuit picks a target point on the path some distance in front of the car and just steers toward it. That's the whole philosophy: give the car a goal, not a correction.

It runs in three steps.

Step 1 — how far ahead to look. The lookahead distance scales with speed:

$$L_d = \text{clip}(k_v \cdot v + L_{\min}, L_{\min}, L_{\max}) = \text{clip}(0.25 \cdot v + 0.8, 0.8, 2.5)$$

The intuition is exactly how humans drive: creeping through a parking lot, you look meters in front of the bumper (tight tracking); at highway speed you look far up the road (smooth steering, no zigzagging after every little curve). Here: $v=0 \rightarrow 0.8$ m, $v=4 \rightarrow 1.8$ m, $v=8 \rightarrow 2.5$ m (clipped).

Step 2 — which point is the target. Find the nearest waypoint, then walk *forward* along the path and return the first one at least L_d away (Euclidean distance from the car; modulo wrap-around because the track is a closed loop). Forward-only is deliberate: the nearest point is usually beside or behind the car, and steering toward that would be a U-turn instruction.

Steps 3 & 4 — the steering law. With α = the angle to the target minus the heading (wrapped to $[-\pi, \pi]$):

$$\delta = \text{atan2}(2 \cdot L \cdot \sin(\alpha), L_d)$$

Where does this come from? There is exactly one circle that passes through the rear axle, is tangent to the car's heading, and reaches the target — the "pursuit arc". Its radius is $R = L_d / (2 \cdot \sin \alpha)$, and a bicycle tracks a circle of radius R with $\tan \delta = L/R$. Stick the two together and the formula above falls out. Sign check: target to the left $\rightarrow \alpha > 0 \rightarrow \delta > 0 \rightarrow$ steer left, consistent with every other convention in this project. I verified it by hand: target dead ahead $\rightarrow 0.0$; 45° left at

4 m lookahead $\rightarrow +0.4169$ rad; mirrored $\rightarrow -0.4169$; a close, steep target saturates at exactly $\pm 35^\circ$ (0.6109 rad).

```
src/Control_Task/Control_Project/bicycle_control/test/test_pure_pursuit.py::test_pure_pursuit_straight PASSED [ 50%]
src/Control_Task/Control_Project/bicycle_control/test/test_pure_pursuit.py::test_pure_pursuit_turn_left PASSED [100%]
```

```
===== 2 passed in 0.78s =====
```

```
"""
High-Level Lateral Steering Controller: Geometric Pure Pursuit.
Calculates steering curvature from lookahead arc geometry.
"""
```

```
import math
import numpy as np
```

```
class PurePursuitController:
    """Adaptive Pure Pursuit lateral controller."""

    def __init__(self, wheelbase=1.25, kv=0.25, l_min=0.8, l_max=2.5,
                 max_steer_rad=math.radians(35.0)):
        self.L = wheelbase
        self.kv = kv
        self.l_min = l_min
        self.l_max = l_max
        self.max_steer_rad = max_steer_rad

    def compute_lookahead(self, v):
        """Adaptive lookahead distance:  $L_d = \text{clip}(kv * v + l_{\min}, l_{\min}, l_{\max})$ ."""
```

```

        ld = self.kv * v + self.l_min
        return float(np.clip(ld, self.l_min, self.l_max))

    def find_target_waypoint(self, x, y, path_points, lookahead):
        """Returns (idx, point) at the look-ahead distance, wrapping the closed loop."""
        if not path_points:
            return 0, None

        n = len(path_points)
        nearest = min(
            range(n),
            key=lambda i: math.hypot(
                path_points[i][0] - x, path_points[i][1] - y
            )
        )

        for k in range(n):
            idx = (nearest + k) % n
            pt = path_points[idx]
            if math.hypot(pt[0] - x, pt[1] - y) >= lookahead:
                return idx, pt

        return nearest, path_points[nearest]

    def compute_steering(self, x, y, yaw, target_pt, lookahead):
        """Computes steering angle in radians using Pure Pursuit geometry."""
        if target_pt is None:
            return 0.0

        dx = target_pt[0] - x
        dy = target_pt[1] - y

        target_yaw = math.atan2(dy, dx)

```

```

alpha = target_yaw - yaw
alpha = math.atan2(math.sin(alpha), math.cos(alpha))

raw_steer = math.atan2(2.0 * self.L * math.sin(alpha), lookahead)
return float(np.clip(raw_steer, -self.max_steer_rad, self.max_steer_rad))

```

Model Predictive Control (Milestone 5.4)

The lateral PID reacts to the present and pure pursuit aims at one future point; MPC is the first controller that actually *simulates the future*. Every 0.1 s it asks: "if I drive this steering-and-throttle sequence for the next $N = 10$ steps, where do I end up, and how bad is that?" It scores candidate sequences, keeps the best one, applies only its **first** step, and re-plans one tick later with fresh measurements. That loop — plan 1 s ahead, execute 0.1 s, repeat — is the "receding horizon."

1. **Decision vector:** 20 numbers, `[ $\delta_0$ ,  $a_0$ ,  $\delta_1$ ,  $a_1$ , ...]` — steering angle and longitudinal acceleration for each of 10 steps. The bounds encode the actuators directly: $\delta \in [-35^\circ, +35^\circ]$, $a \in [-4, +4] \text{ m/s}^2$. The optimizer physically cannot command an impossible steering angle — that's what "constrained optimization" buys you over the other controllers.
2. **Prediction model:** the same forward-Euler extended kinematic bicycle from Milestone 2, so position uses the *o/d* speed and yaw, and v is an explicit state integrated with a_k . I also subtract drag in the prediction (`$c_{\text{drag}} \cdot v^2 + c_{\text{roll}} \cdot v$`) so the model matches the simulator's resistance dynamics — the MPC then doesn't systematically over-predict speed on straights. Any residual model error gets absorbed anyway by re-planning from measured state every tick.
3. **Cost — the Frenet frame:** the tracking error is projected onto the reference path's own axes: the component along the path tangent (`$e_{\text{long}}$` = "ahead/behind") is separate from the component along the normal (`$cte$` = "left/right"), using the reference waypoint's yaw. Without this, a car 1 m behind the reference but perfectly on the line would be punished for lateral error it doesn't have. The seven weighted terms: `$w_{\text{lat}}=30$` (stay on the line — the dominant term), `$w_{\text{yaw}}=10$` (point along the line), `$w_{\text{dsteer}}=6$` (don't jerk the wheel

— this penalizes $\delta k - \delta k_{-1}$ with δk_{-1} initialized from the *actual* current steering, so commands stay smooth across ticks), plus small penalties on speed error, steering magnitude, and acceleration as regularizers.

4. **Warm start:** the previous solution shifted one step becomes the initial guess, so SLSQP usually converges within its budget of 25 iterations instead of starting from zero every tick.
5. **Extraction:** take `u*[0]` as steering and map `u*[1]` from m/s^2 to normalized throttle via `a/k_a` — the MPC speaks physics, the car speaks -1..1.

```
youssef_ahmed@DESKTOP-OK79IR3:~$ cd ~/ros2_ws/ros2_ws
source install/setup.bash
python3 -m pytest src/Control_Task/Control_Project/bicycle_control/test/test_mpc.py -v
colcon-core-0.21.2
collected 1 item

src/Control_Task/Control_Project/bicycle_control/test/test_mpc.py::test_mpc_straight_tracking PASSED [100%]

===== 1 passed in 0.30s =====
```

```
"""
```

High-Level Lateral Steering Controller: Extended Kinematic Bicycle MPC.

Solves a constrained non-linear program over prediction horizon N using SciPy, optimizing steering angle and longitudinal acceleration (mapped to throttle).

```
"""
```

```
import math # noqa: F401
import numpy as np # noqa: F401
from scipy.optimize import minimize # noqa: F401
```

```
class KinematicBicycleMPC:
```

```
"""Nonlinear Model Predictive Control for an Extended Kinematic Bicycle Model.
```

```
Optimizes future control sequences  $u = [\delta_k, a_k]$  where steering angle  $\delta_k$ 
```

```
and longitudinal acceleration  $a_k$  (mapped to throttle effort) are the control inputs,
```

```
forward-simulating a 4-state extended kinematic bicycle model  $x = [x, y, \theta, v]^T$ .
```

```
"""
```

```
def __init__(self, wheelbase=1.25, dt=0.1, horizon=10,
              max_steer_rad=math.radians(35.0), k_a=4.0,
              max_accel=None, max_brake=None, c_drag=0.005, c_roll=0.05):
```

```
    self.L = wheelbase
```

```
    self.dt = dt
```

```
    self.N = horizon
```

```
    self.max_steer_rad = max_steer_rad
```

```
    self.k_a = float(max_accel if max_accel is not None else k_a)
```

```
    self.c_drag = c_drag
```

```
    self.c_roll = c_roll
```

```
    # Weights: heavily penalize lateral CTE, heading error, and steering rate
```

```
    self.w_lat = 30.0
```

```
    self.w_long = 1.0
```

```
    self.w_yaw = 10.0
```

```
    self.w_v = 1.0
```

```
    self.w_steer = 0.2
```

```
    self.w_dsteer = 6.0
```

```
    self.w_accel = 0.1
```

```
    self.last_u = np.zeros(2 * self.N)
```

```

def solve(self, x0, ref_trajectory, current_steering=0.0):
    """Solves MPC optimization problem over horizon N."""
    N_eff = min(self.N, len(ref_trajectory))
    if N_eff < 2:
        return 0.0, 0.0

    bounds = [(-self.max_steering_rad, self.max_steering_rad),
               (-self.k_a, self.k_a)] * N_eff

    def objective(u):
        cost = 0.0
        x, y, yaw, v = x0
        prev_delta = current_steering

        for k in range(N_eff):
            delta_k = u[2 * k]
            a_k = u[2 * k + 1]

            x += v * math.cos(yaw) * self.dt
            y += v * math.sin(yaw) * self.dt
            yaw += (v / self.L) * math.tan(delta_k) * self.dt

            # Match simulation resistance dynamics exactly
            a_resistance = self.c_drag * \
                (v ** 2) + self.c_roll * v if v > 0 else 0.0
            v += (a_k - a_resistance) * self.dt

            x_ref, y_ref, yaw_ref, v_ref = ref_trajectory[k]

            dx = x - x_ref
            dy = y - y_ref

            cte = -dx * math.sin(yaw_ref) + dy * math.cos

```

```

(yaw_ref)
    e_long = dx * math.cos(yaw_ref) + dy * math.s
in(yaw_ref)

    e_yaw = yaw - yaw_ref
    e_yaw = math.atan2(math.sin(e_yaw), math.cos

(e_yaw))

    e_v = v - v_ref
    d_steer = (delta_k - prev_delta) / self.dt

    cost += self.w_lat * (cte ** 2)
    cost += self.w_long * (e_long ** 2)
    cost += self.w_yaw * (e_yaw ** 2)
    cost += self.w_v * (e_v ** 2)
    cost += self.w_steer * (delta_k ** 2)
    cost += self.w_dsteer * (d_steer ** 2)
    cost += self.w_accel * (a_k ** 2)

    prev_delta = delta_k

    return cost

u_init = np.zeros(2 * N_eff)
if len(self.last_u) >= 2 * N_eff:
    u_init[:-2] = self.last_u[2:2 * N_eff]
    u_init[-2:] = self.last_u[2 * N_eff - 2: 2 * N_ef
f]

res = minimize(
    objective,
    u_init,
    bounds=bounds,
    method='SLSQP',
    options={'maxiter': 25, 'ftol': 1e-3}
)

```

```

self.last_u = np.zeros(2 * self.N)
self.last_u[:2 * N_eff] = res.x

delta_cmd = float(res.x[0])
accel_cmd = float(res.x[1])
throttle_cmd = float(np.clip(accel_cmd / self.k_a, -
1.0, 1.0))

return delta_cmd, throttle_cmd

```

Milestone 5.5: Lap analyzer

This is our judge. It sits quietly, watching `/path` and `/state`, taking notes without ever touching the steering wheel or interfering with the driver.

For every tick of the clock, the judge measures exactly how far off the race line the car is drifting—keeping track of every inch to the left or right, along with speed and heading errors. At the end of every run, the judge prints out a clean summary report and saves the data straight to CSV.

```

"""
Lap Analyzer Node:
Performance evaluation, real-time telemetry, lap timing, CSV
logging, and RViz HUD visualization.
"""

import json
import math
import os

import numpy as np

```

```

import rclpy # noqa: F401 # type: ignore[import-not-found]
from rclpy.node import Node # type: ignore[import-not-found]
from nav_msgs.msg import Path, Odometry # type: ignore[import-not-found]
from std_msgs.msg import String, Float32 # type: ignore[import-not-found]
from geometry_msgs.msg import Point # type: ignore[import-not-found]
# type: ignore[import-not-found]
from visualization_msgs.msg import Marker, MarkerArray

class LapAnalyzer(Node):
    def __init__(self):
        super().__init__('lap_analyzer')
        self.get_logger().info('Initializing Lap Analyzer Node...')

        # Parameters
        self.declare_parameter('log_file', '')
        self.log_file = str(self.get_parameter('log_file').value)

        # Subscriptions & Publishers
        self.path_sub = self.create_subscription(
            Path, '/path', self.path_callback, 10)
        self.state_sub = self.create_subscription(
            Odometry, '/state', self.state_callback, 10)

        self.metrics_pub = self.create_publisher(String, '/lap/metrics', 10)
        self.viz_pub = self.create_publisher(
            MarkerArray, '/lap/visualization', 10)

        self.cte_pub = self.create_publisher(Float32, '/lap/telemetry/cte', 10)

```

```

        self.speed_pub = self.create_publisher(Float32, '/telemetry/speed', 10)
        self.heading_err_pub = self.create_publisher(
            Float32, '/telemetry/heading_err_deg', 10)
        self.lap_time_pub = self.create_publisher(
            Float32, '/telemetry/lap_time', 10)

        self.path_points = []
        self.path_cum_dist = []
        self.track_length = 0.0
        self.path_received = False

        self.start_sim_time = None
        self.last_state_time = None
        self.lap_start_time = None

        self.lap_count = 0
        self.last_s = 0.0
        self.total_distance = 0.0
        self.lap_distance = 0.0
        self.last_xy = None

        self.current_lap_time = 0.0
        self.last_lap_time = None
        self.best_lap_time = None
        self.lap_times = []

        self.lap_ctes = []
        self.lap_heading_errors = []
        self.lap_speeds = []

        self.global_ctes = []
        self.global_max_speed = 0.0

        self.current_cte = 0.0
        self.current_heading_err = 0.0

```

```

self.current_speed = 0.0
self.proj_xy = (0.0, 0.0)

self.timer = self.create_timer(0.1, self.publish_telemetry)

def path_callback(self, msg: Path):
    """Processes received path and precomputes cumulative distance."""
    if self.path_received and len(msg.poses) == len(self.path_points):
        return

    pts = []
    for p in msg.poses:
        x = p.pose.position.x
        y = p.pose.position.y
        qz = p.pose.orientation.z
        qw = p.pose.orientation.w
        yaw = 2.0 * math.atan2(qz, qw)
        pts.append((x, y, yaw))

    if len(pts) < 2:
        return

    self.path_points = pts
    cum = [0.0]
    for i in range(1, len(pts)):
        d = math.hypot(pts[i][0] - pts[i - 1][0],
                       pts[i][1] - pts[i - 1][1])
        cum.append(cum[-1] + d)

    self.path_cum_dist = cum
    self.track_length = cum[-1]
    self.path_received = True
    self.get_logger().info(

```

```

        f"Lap Analyzer: Loaded path with {len(pts)} waypoints, "
        f"perimeter: {self.track_length:.2f} m"
    )

    def state_callback(self, msg: Odometry):
        """Processes vehicle odometry and updates progress, lap timing, and errors."""
        now_sec = msg.header.stamp.sec + msg.header.stamp.nanosec * 1e-9

        if self.start_sim_time is None:
            self.start_sim_time = now_sec
            self.lap_start_time = now_sec

        x = msg.pose.pose.position.x
        y = msg.pose.pose.position.y
        qz = msg.pose.pose.orientation.z
        qw = msg.pose.pose.orientation.w
        yaw = 2.0 * math.atan2(qz, qw)
        v = msg.twist.twist.linear.x

        self.current_speed = v
        self.global_max_speed = max(self.global_max_speed, v)

        if self.last_xy is not None:
            step_d = math.hypot(x - self.last_xy[0], y - self.last_xy[1])
            self.total_distance += step_d
            self.lap_distance += step_d

        self.last_xy = (x, y)

        if not self.path_received or len(self.path_points) < 2:
            return

```

```

        proj_x, proj_y, s, cte, heading_err = self.project_to_
_path(x, y, yaw)
        self.proj_xy = (proj_x, proj_y)
        self.current_cte = cte
        self.current_heading_err = heading_err

        abs_cte = abs(cte)
        self.lap_ctes.append(abs_cte)
        self.lap_heading_errors.append(abs(heading_err))
        self.lap_speeds.append(v)
        self.global_ctes.append(abs_cte)

        self.current_lap_time = now_sec - self.lap_start_time

        # False lap trigger guard: Require driving >= 90% of
        track length before crossing
        if self.track_length > 5.0 and v > 0.1 and self.lap_d
istance > 0.9 * self.track_length:
            if self.last_s > 0.75 * self.track_length and s <
0.25 * self.track_length:
                ds_total = (self.track_length - self.last_s)
+ s
                dt_step = max(
                    now_sec - (self.last_state_time or now_se
c), 1e-4)
                frac = (self.track_length - self.last_s) / ma
x(ds_total, 1e-4)
                t_crossing = (self.last_state_time or now_se
c) + frac * dt_step

                lap_duration = t_crossing - self.lap_start_ti
me
                self.record_lap_completion(lap_duration, now_
sec)
                self.lap_start_time = t_crossing

```

```

self.last_s = s
self.last_state_time = now_sec

def project_to_path(self, x, y, yaw):
    """Finds closest segment and projects (x, y) to compute exact orthogonal CTE."""
    pts = self.path_points
    n = len(pts)

    min_dist_sq = float('inf')
    nearest_idx = 0
    for i in range(n):
        dx = pts[i][0] - x
        dy = pts[i][1] - y
        d_sq = dx * dx + dy * dy
        if d_sq < min_dist_sq:
            min_dist_sq = d_sq
            nearest_idx = i

    best_dist = float('inf')
    best_proj = (pts[nearest_idx][0], pts[nearest_idx][1])

    best_s = self.path_cum_dist[nearest_idx]
    best_seg_yaw = pts[nearest_idx][2]
    best_signed_cte = 0.0

    candidate_segments = [
        ((nearest_idx - 1) % n, nearest_idx),
        (nearest_idx, (nearest_idx + 1) % n)
    ]
    for prev_i, next_i in candidate_segments:
        x1, y1, yaw1 = pts[prev_i]
        x2, y2, _ = pts[next_i]
        dx = x2 - x1
        dy = y2 - y1
        seg_len_sq = dx * dx + dy * dy

```

[1])

```

        if seg_len_sq < 1e-6:
            continue

        t = max(0.0, min(1.0, ((x - x1) * dx + (y - y1) *
dy) / seg_len_sq))
        px = x1 + t * dx
        py = y1 + t * dy
        dist = math.hypot(x - px, y - py)

        if dist < best_dist:
            best_dist = dist
            best_proj = (px, py)
            best_s = self.path_cum_dist[prev_i] + t * mat
h.sqrt(seg_len_sq)
            best_seg_yaw = math.atan2(dy, dx)

            cross = dx * (y - y1) - dy * (x - x1)
            best_signed_cte = math.copysign(dist, cross)

        heading_err = math.atan2(
            math.sin(yaw - best_seg_yaw), math.cos(yaw - best
_seg_yaw)
        )

        return best_proj[0], best_proj[1], best_s, best_signe
d_cte, heading_err

    def record_lap_completion(self, lap_duration, now_sec):
        """Records finished lap, prints summary, and appends
stats to CSV."""
        self.lap_count += 1
        self.last_lap_time = lap_duration
        self.lap_times.append(lap_duration)

        if self.best_lap_time is None or lap_duration < self.
best_lap_time:

```

```

        self.best_lap_time = lap_duration

        ctes_arr = np.array(
            self.lap_ctes) if self.lap_ctes else np.array([0.
0])

        speeds_arr = np.array(
            self.lap_speeds) if self.lap_speeds else np.array
([0.0])

        mean_cte = float(np.mean(ctes_arr))
        max_cte = float(np.max(ctes_arr))
        rms_cte = float(np.sqrt(np.mean(ctes_arr ** 2)))

        mean_speed = float(np.mean(speeds_arr))
        max_speed = float(np.max(speeds_arr))

        summary = (
            f"\n{'='*50}\n"
            f"LAP {self.lap_count} COMPLETED\n"
            f"\n{'='*50}\n"
            f"Lap Time:      {lap_duration:.2f} s\n"
            f"Best Lap:      {self.best_lap_time:.2f} s\n"
            f"Mean CTE:      {mean_cte:.4f} m\n"
            f"Max CTE:      {max_cte:.4f} m\n"
            f"RMS CTE:      {rms_cte:.4f} m\n"
            f"Mean Speed:    {mean_speed:.2f} m/s\n"
            f"Max Speed:     {max_speed:.2f} m/s\n"
            f"Total Dist:    {self.total_distance:.1f} m\n"
            f"\n{'='*50}"
        )
        self.get_logger().info(summary)

        # Reset lap distance buffer
        self.lap_distance = 0.0

        # Export metrics to CSV file if parameter is specifie

```

d

```
if self.log_file:
    new = not os.path.isfile(self.log_file)
    with open(self.log_file, 'a') as f:
        if new:
            f.write(
                'lap,time_s,mean_cte,max_cte,rms_cte,
mean_speed,max_speed\n')
            f.write(
                f'{self.lap_count},{lap_duration:.3f},{me
an_cte:.4f},{max_cte:.4f},'
                f'{rms_cte:.4f},{mean_speed:.3f},{max_spe
ed:.3f}\n'
            )

        self.lap_ctes.clear()
        self.lap_heading_errors.clear()
        self.lap_speeds.clear()

def publish_telemetry(self):
    """Periodically publishes numerical telemetry and RVi
z visual markers at 10 Hz."""
    self.cte_pub.publish(Float32(data=float(self.current_
cte)))
    self.speed_pub.publish(Float32(data=float(self.curren
t_speed)))
    self.heading_err_pub.publish(
        Float32(data=float(math.degrees(self.current_head
ing_err)))
    )
    self.lap_time_pub.publish(Float32(data=float(self.cur
rent_lap_time)))

    ctes_arr = np.array(
        self.lap_ctes) if self.lap_ctes else np.array([0.
0])
```

```

        live_rms_cte = float(np.sqrt(np.mean(ctes_arr ** 2)))

        g = np.array(self.global_ctes) if self.global_ctes else np.array([0.0])

        telemetry = {
            "lap": self.lap_count + 1,
            "laps_completed": self.lap_count,
            "current_lap_time": round(self.current_lap_time,
2),
            "last_lap_time": round(self.last_lap_time, 2) if
self.last_lap_time else None,
            "best_lap_time": round(self.best_lap_time, 2) if
self.best_lap_time else None,
            "speed": round(self.current_speed, 2),
            "top_speed": round(self.global_max_speed, 2),
            "current_cte": round(self.current_cte, 4),
            "rms_cte": round(live_rms_cte, 4),
            "mean_cte_all": round(float(g.mean()), 4),
            "max_cte_all": round(float(g.max()), 4),
            "rms_cte_all": round(float(np.sqrt((g ** 2).mean
())), 4),
            "heading_err_deg": round(math.degrees(self.curren
t_heading_err), 2)
        }
        self.metrics_pub.publish(String(data=json.dumps(telem
etry)))
        self.publish_rviz_markers(telemetry)

    def publish_rviz_markers(self, telemetry=None):
        """Renders start gate, error whisker, and on-screen H
UD text in RViz."""
        ma = MarkerArray()
        now = self.get_clock().now().to_msg()

        if self.path_points:

```

```

    p0 = self.path_points[0]
    gate = Marker()
    gate.header.frame_id = 'map'
    gate.header.stamp = now
    gate.ns = 'start_gate'
    gate.id = 0
    gate.type = Marker.CYLINDER
    gate.action = Marker.ADD
    gate.pose.position.x = p0[0]
    gate.pose.position.y = p0[1]
    gate.pose.position.z = 0.5

    # Align start gate orientation perpendicular to t
    rack heading (p0[2])
    gate.pose.orientation.z = math.sin(p0[2] / 2.0)
    gate.pose.orientation.w = math.cos(p0[2] / 2.0)

    gate.scale.x = 0.1
    gate.scale.y = 1.2
    gate.scale.z = 1.0
    gate.color.r = 0.1
    gate.color.g = 0.9
    gate.color.b = 0.2
    gate.color.a = 0.7
    ma.markers.append(gate)

    if self.last_xy is not None and self.proj_xy is not N
one:
        whisker = Marker()
        whisker.header.frame_id = 'map'
        whisker.header.stamp = now
        whisker.ns = 'cte_whisker'
        whisker.id = 1
        whisker.type = Marker.LINE_STRIP
        whisker.action = Marker.ADD
        whisker.scale.x = 0.05

```

```

        p_vehicle = Point(
            x=float(self.last_xy[0]), y=float(self.last_x
y[1]), z=0.0)
        p_proj = Point(x=float(self.proj_xy[0]), y=float(
            self.proj_xy[1]), z=0.0)
        whisker.points = [p_vehicle, p_proj]

        abs_cte = abs(self.current_cte)
        if abs_cte < 0.2:
            whisker.color.r, whisker.color.g, whisker.col
or.b = 0.0, 1.0, 0.0
        elif abs_cte > 0.5:
            whisker.color.r, whisker.color.g, whisker.col
or.b = 1.0, 0.0, 0.0
        else:
            whisker.color.r, whisker.color.g, whisker.col
or.b = 1.0, 1.0, 0.0
        whisker.color.a = 1.0

        ma.markers.append(whisker)

    if telemetry is not None and self.path_points:
        hud = Marker()
        hud.header.frame_id = 'map'
        hud.header.stamp = now
        hud.ns = 'telemetry_hud'
        hud.id = 2
        hud.type = Marker.TEXT_VIEW_FACING
        hud.action = Marker.ADD

        p0 = self.path_points[0]
        hud.pose.position.x = p0[0]
        hud.pose.position.y = p0[1]
        hud.pose.position.z = 5.0

```

```

        hud.scale.z = 0.8
        hud.color.r, hud.color.g, hud.color.b, hud.color.
a = 1.0, 1.0, 1.0, 1.0

        best = f"{telemetry['best_lap_time']}s" if teleme
try['best_lap_time'] else "N/A"
        hud.text = (
            f"LAP {telemetry['lap']}\n"
            f"Time: {telemetry['current_lap_time']:.1f}s
| Best: {best}\n"
            f"Speed: {telemetry['speed']:.1f} m/s\n"
            f"CTE: {telemetry['current_cte']:.3f} m\n"
            f"RMS CTE: {telemetry['rms_cte']:.3f} m"
        )

        ma.markers.append(hud)

    self.viz_pub.publish(ma)

def main(args=None):
    rclpy.init(args=args)
    analyzer = LapAnalyzer()
    try:
        rclpy.spin(analyzer)
    except KeyboardInterrupt:
        pass
    finally:
        analyzer.destroy_node()
        if rclpy.ok():
            rclpy.shutdown()

if __name__ == '__main__':
    main()

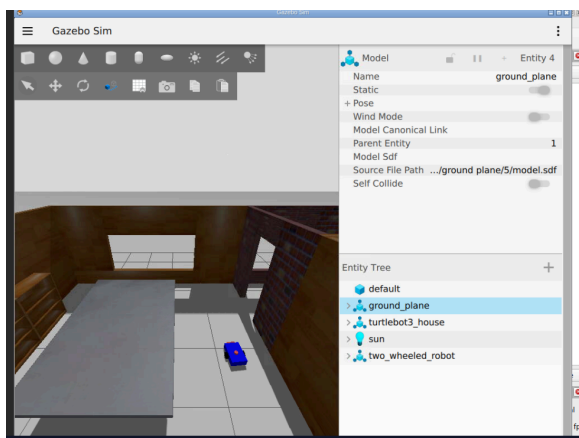
```

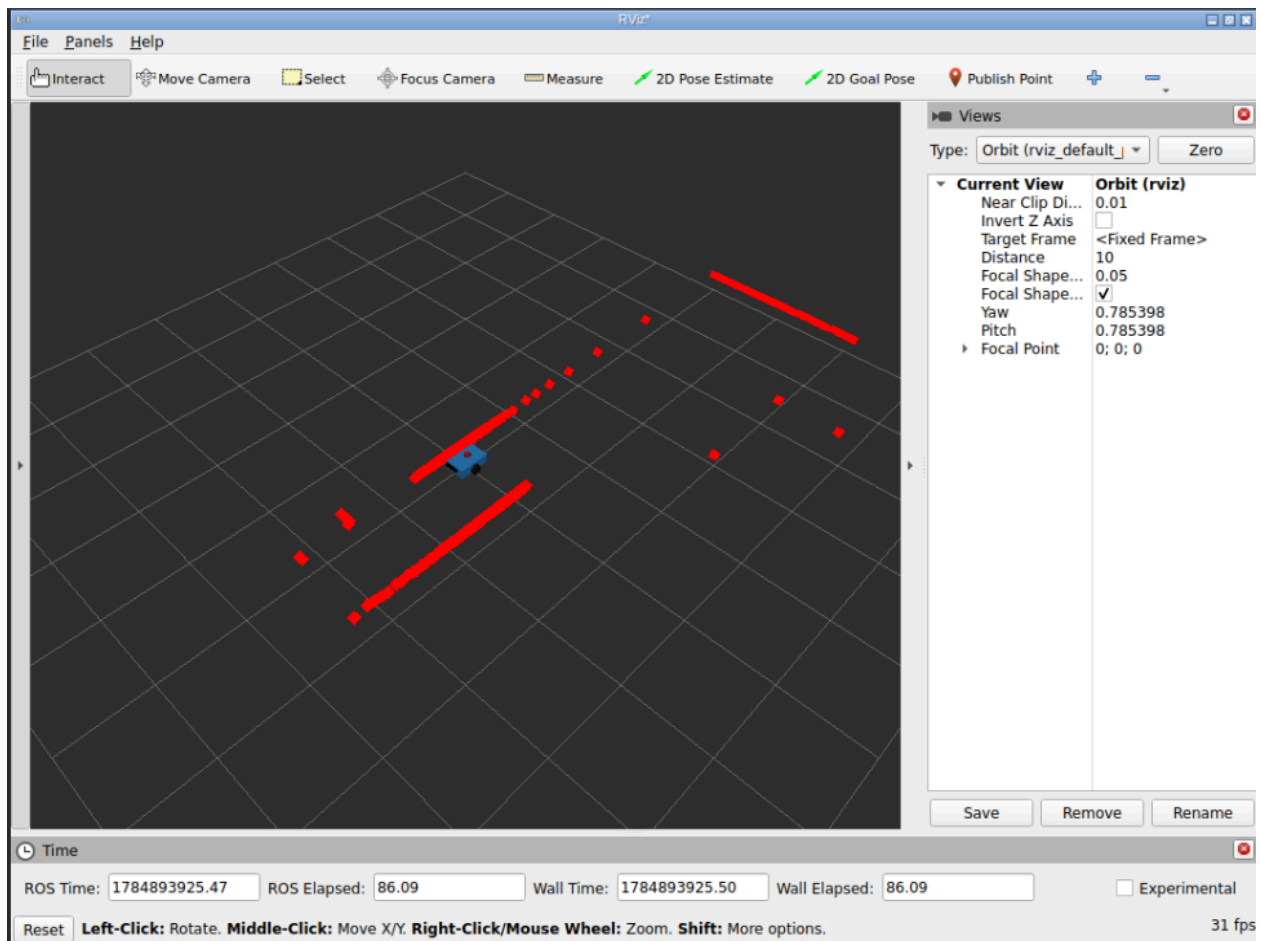
Milestone 6: Gazebo

so as you know the Gazebo is a visualization tool. So that rises the next question what is the difference between it and Rviz. And what does Gazebo exactly do. Instead of just drawing colored lines, boxes, and trajectories on a screen, Gazebo simulates actual physical environment interactions:

- **Gravity and Mass:** Objects have weight, inertia, and centers of gravity.
- **Rigid-Body Dynamics:** Collision forces occur when two objects hit each other.
- **Tire Mechanics & Friction:** Wheels interact with surface friction coefficients, slipping or gripping depending on vehicle load and speed.
- **Sensor Simulation:** Real-time generation of raw sensor data with realistic noise (3D LiDAR point clouds, camera streams, IMU accelerations).

In short: Gazebo tests whether your autonomous system can survive in a physical universe. Actually Gazebo is not just a visualization tool it is a complete physics simulator.

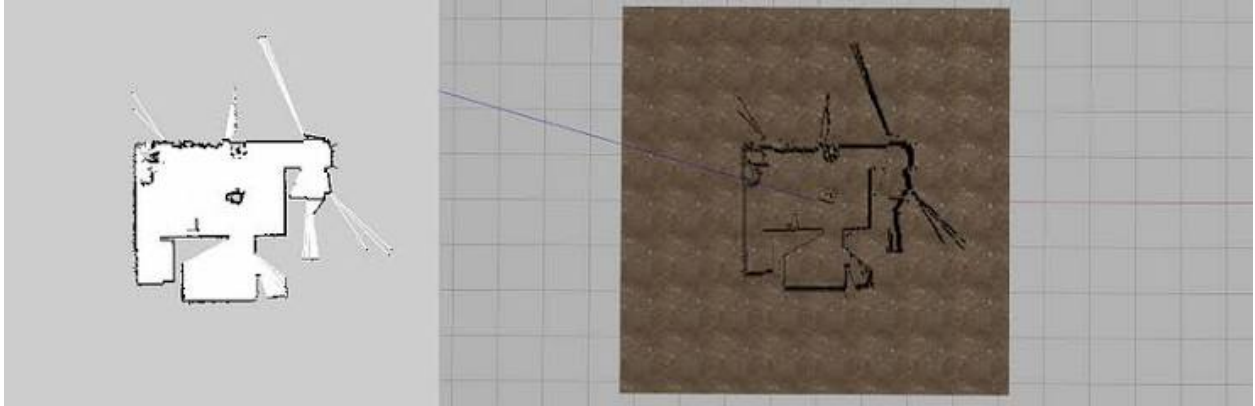




Why do we need Gazebo if we already have RViz?

The short answer is **kinematics vs. dynamics**.

RViz (Data Visualization):



Displays what the robot THINKS is happening (Odometry, Transforms, Waypoints, Path lines).

Motion is mathematical & ideal (Kinematic: x , y , θ update instantly via ODEs)

Wheels turn mathematically with zero resistance.

No collisions or body roll dynamics.

Gazebo (Physics Simulator):

Simulates what ACTUALLY happens in the physical world.

- Motion is subject to:
 - Mass & Inertia
 - Tire slip & friction
 - Motor torque limits
 - Aerodynamic drag
 - Vehicle can slide, flip, or crash into barriers.

How to install and run Gazebo:

```
# 1. Update system package index
sudo apt update
```

```
# 2. Install Gazebo simulator & ROS 2 integration bridges
```

```
sudo apt install -y ros-humble-ros-gz ros-humble-gazebo-ros-p  
kgs
```

```
# 3. Install ros2_control Gazebo plugins (for joint & torque  
control)
```

```
sudo apt install -y ros-humble-gazebo-ros2-control
```

```
# 4. Install xacro (if not installed yet)
```

```
sudo apt install -y ros-humble-xacro
```

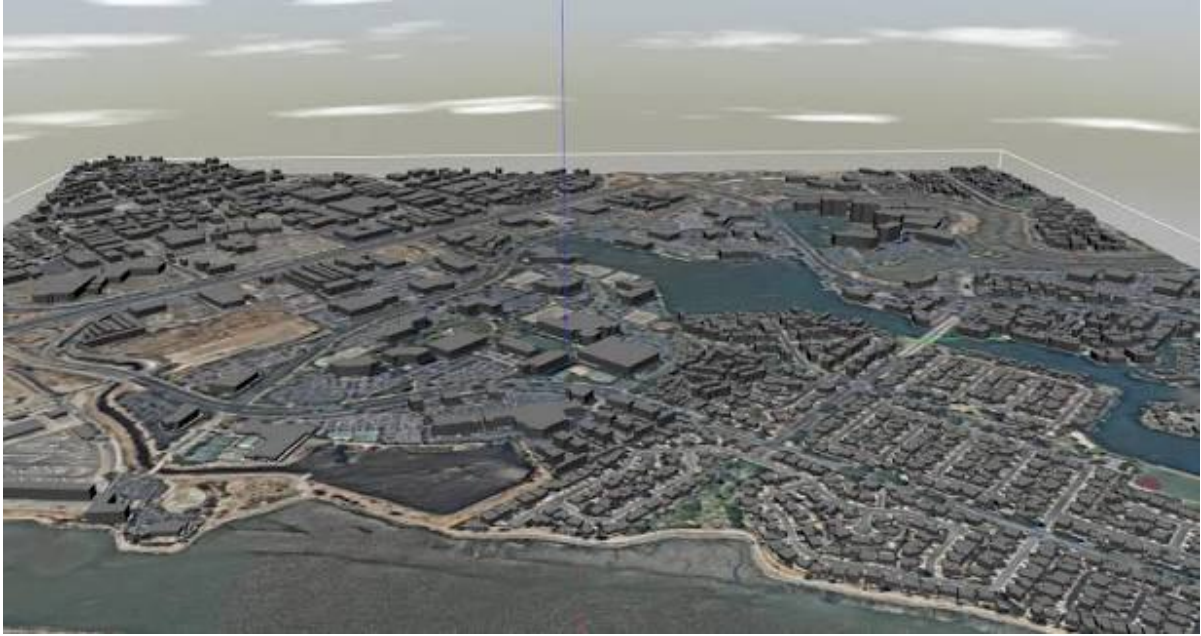
```
# 5. Verify installation by launching an empty Gazebo world  
gazebo
```

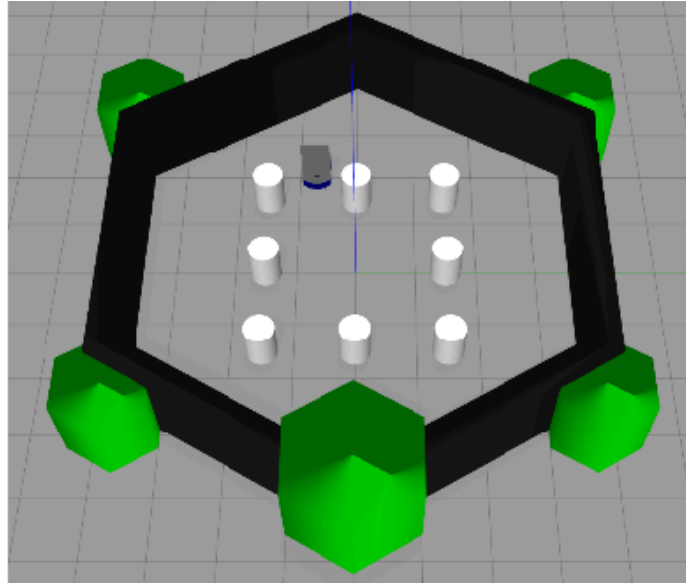
Available Maps & Worlds in Gazebo

- **Empty World (`empty.world`)**: A flat infinite plane with basic sunlight and zero obstacles. Great for initial powertrain calibration and open-loop steering tests.
- **Shapes / Playground (`shapes.world`)**: Contains basic geometric solids (cubes, spheres, cylinders) to test collision sensors like LiDAR.

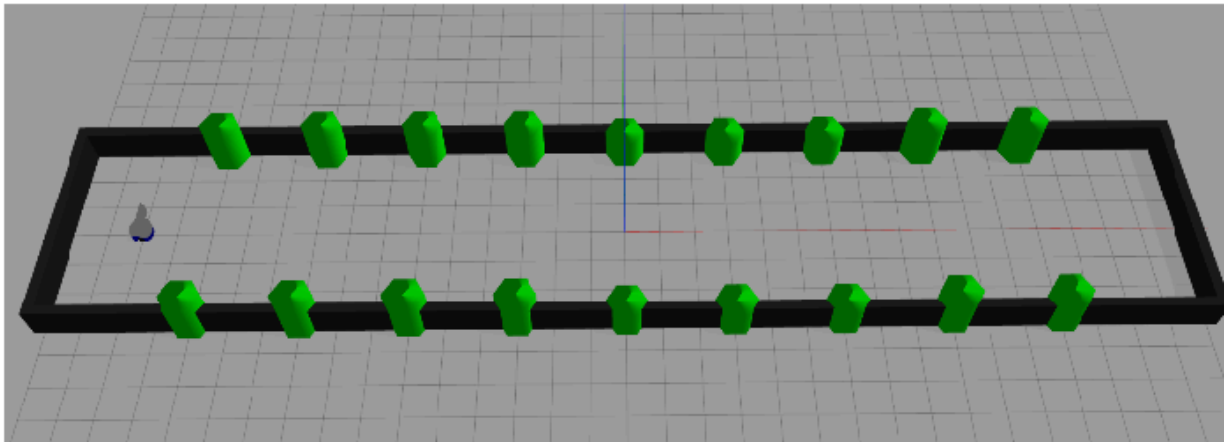
B. Autonomous Racing & Track Worlds

- **Racecar Track Worlds (`track_environment` / F1TENTH / AWS DeepRacer)**: Custom `.world` or `.sdf` files containing 3D asphalt surfaces, concrete barriers, and cone-marked turns





(a)



(b)

Video Link:

https://drive.google.com/drive/folders/1sEosf_EoRhrUmlJPqEHOWStZc7Otv-0p?usp=sharing